

Persistent and Secure WebSocket Chat

Persistence, Encryption, Integrity and Authenticity

Indian Institute of Technology Bhilai

14th of August, 2026

From Real-Time Chat to Persistent Chat

Last week: messages were delivered in real time using WebSockets.



New question:

- ▶ Where are yesterday's messages?
- ▶ What happens if the server restarts?
- ▶ Can someone reading the database see the messages?
- ▶ Can someone modify a stored message?
- ▶ Who created this message?

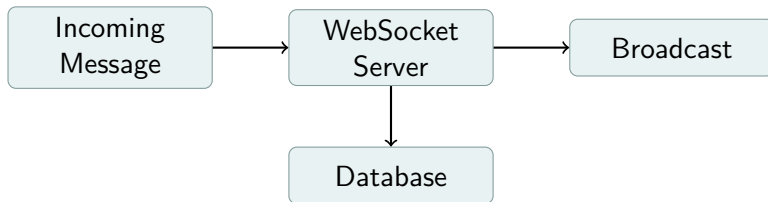
What Is Missing From Our Chat?

Requirement	Question
Persistence	Can I retrieve the message later?
Confidentiality	Can an unauthorized person read it?
Integrity	Was the stored message modified?
Authenticity	Who created or signed the message?

Goal: turn the previous WebSocket chat into a system that is **persistent and secure**.

Persistence: Store the Message

Instead of only broadcasting a message:

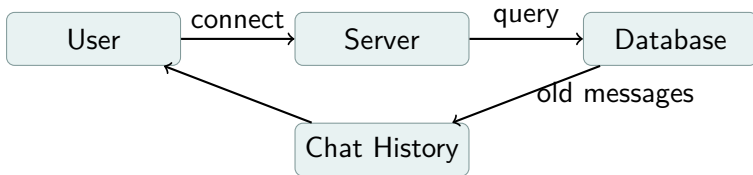


A simple message record can contain:

- ▶ `message_id`
- ▶ `room_id`
- ▶ `sender_id`
- ▶ `message`
- ▶ `timestamp`

Loading Chat History

When a user joins:



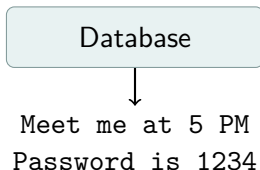
After history is loaded, the WebSocket remains open for new messages.

Key idea

WebSocket provides **real-time communication**; the database provides **persistence**.

Should We Store Plaintext?

Suppose the database contains:

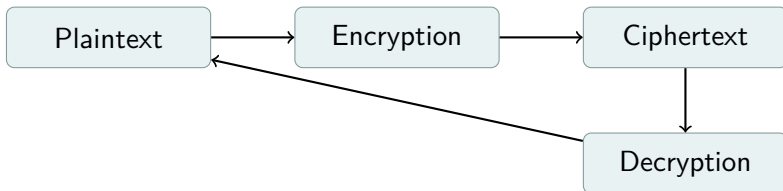


Anyone who gains unauthorized access to the database can read the messages.

Plaintext $\longrightarrow$ **Encryption** $\longrightarrow$ **Ciphertext**

- ▶ Store ciphertext instead of plaintext.
- ▶ Only a party with the required key can recover the message.

Encryption and Decryption



Symmetric encryption

- ▶ The same secret key is used to encrypt and decrypt.
- ▶ The key must be protected.
- ▶ Use a standard cryptographic library; do not implement AES yourself.

For the lab

Use an authenticated encryption mode such as **AES-GCM**.

Encryption Does Not Solve Everything

Suppose the database contains:

```
ciphertext = ABC123...
```

An attacker changes it:

```
ciphertext = XYZ999...
```

Encryption answers:

“Can someone read the message without the key?”

But we also need to answer:

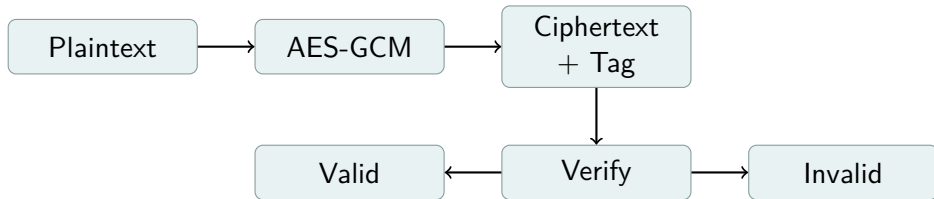
“Was the encrypted data modified?”

We need integrity

A secure messaging system must detect unauthorized modification.

Integrity: Detect Modification

Authenticated encryption gives us ciphertext together with authentication data.



- ▶ Valid authentication data → continue to decrypt.
- ▶ Invalid authentication data → reject the message.

How Do We Know Who Sent It?

Suppose the database says:

```
sender = Alice  
message = "Transfer 5000"
```

Can we prove that Alice actually created the message?

Digital Signature

A sender uses a **private key** to sign data.

Others use the corresponding **public key** to verify the signature.

Private Key $\longrightarrow$ Sign $\longrightarrow$ Message + Signature

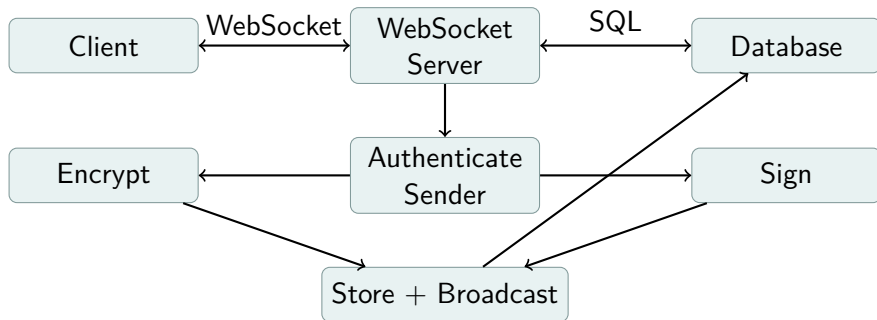
Message + Signature + **Public Key** $\longrightarrow$ Verify

Encryption vs Integrity vs Signature

Mechanism	Main question
Database persistence	Can I retrieve it later?
Encryption	Can an unauthorized person read it?
Integrity / authentication tag	Was the protected data modified?
Digital signature	Can I verify who signed the data?

A secure chat needs all four properties.

Final Secure Chat Architecture



When reading history:

Retrieve → Verify → Decrypt → Display

Starting Point: Last Week's Chat

Extend the existing WebSocket chat instead of creating a new application.

```
async def chat(websocket):  
    clients.add(websocket)  
  
    async for message in websocket:  
        for client in clients:  
            await client.send(message)  
  
    clients.remove(websocket)
```

New responsibility:

Receive → Store → Broadcast

Create a SQLite Database

A lightweight database is enough for the lab.

```
import sqlite3

conn = sqlite3.connect("chat.db")

conn.execute("""
CREATE TABLE IF NOT EXISTS messages (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    room_id TEXT,
    sender TEXT,
    message TEXT,
    timestamp DATETIME DEFAULT CURRENT_TIMESTAMP
)
""")

conn.commit()
```

Idea: every message becomes a persistent record.

Save Incoming Messages

```
def save_message(room_id, sender, message):  
    conn.execute(  
        """  
        INSERT INTO messages(room_id, sender, message)  
        VALUES (?, ?, ?)  
        """,  
        (room_id, sender, message)  
    )  
    conn.commit()
```

Then the WebSocket handler becomes:

```
async for message in websocket:  
    save_message(room_id, sender, message)  
    await broadcast(message)
```

Load Previous Messages

```
def get_history(room_id):  
    cursor = conn.execute(  
        """  
        SELECT sender, message, timestamp  
        FROM messages  
        WHERE room_id = ?  
        ORDER BY id  
        """,  
        (room_id,) )  
    return cursor.fetchall()
```

When a client joins:

```
history = get_history(room_id)  
  
for row in history:  
    await websocket.send(serialize(row))
```

Result: user can see previous chat messages.

Encrypt Before Storing

Use a standard cryptographic library.

```
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
import os

key = AESGCM.generate_key(bit_length=256)
aes = AESGCM(key)

nonce = os.urandom(12)

ciphertext = aes.encrypt(
    nonce,
    message.encode(),
    None
)
```

Important

Do not write your own encryption algorithm. Use a standard cryptographic implementation.

Decrypt When Reading

```
plaintext = aes.decrypt(  
    nonce,  
    ciphertext,  
    None  
)  
  
message = plaintext.decode()
```

The flow is:

Database → Ciphertext → Decrypt → Plaintext → Client

Notice

The database should contain ciphertext, not the original plaintext message.

AES-GCM Also Detects Tampering

Store the values required for decryption:

```
ciphertext  
nonce
```

If the ciphertext is modified:

```
plaintext = aes.decrypt(  
    nonce,  
    modified_ciphertext,  
    None  
)
```

The operation should fail with an authentication error.

Demo

Change a ciphertext value directly in the database and try to read the message again.

Digital Signature: Signing

Use a public/private key pair for the sender.

```
signature = private_key.sign(  
    message.encode()  
)
```

Store the signature with the message metadata.

```
sender  
ciphertext  
nonce  
signature  
timestamp
```

The sender's **private key must never be shared.**

Digital Signature: Verification

The receiver uses the sender's public key.

```
public_key.verify(  
    signature,  
    message.encode()  
)
```

Possible outcomes:

Result		Meaning
Verification succeeds	success	Signature matches the message.
Verification fails		Message or signature was modified, or the wrong key was used.

Complete Message Flow

Sending

Message → Authenticate → Encrypt → Sign → Store → Broadcast

Receiving history

Retrieve → Verify Signature → Verify/Decrypt → Display

A stored record can conceptually contain:

```
message_id  
room_id  
sender_id  
ciphertext  
nonce  
signature  
timestamp
```

Lab Assignment: Secure Persistent Group Chat

Goal: Extend your previous WebSocket group chat into a persistent and secure messaging system.

Mandatory requirements

1. Messages are stored in a database.
2. A user receives previous chat history.
3. Messages are not stored as plaintext.
4. The system detects modification of a stored message.
5. Each sender has a signing key pair.
6. Messages contain a sender signature and the signature is verified.

Submission

- ▶ Source code, pdf report and contribution report of each member.
- ▶ Working Client URL hosted on your allotted system.
- ▶ Public GitHub link with all members as collaborators.
- ▶ Screenshots or short demo video consists of demonstration of persistence, tamper detection and signature verification.

IS IT REALLY SECURED?